JAVASCRIPT BASICS

A beginner-friendly guide to the language of interactive web pages

Topics include syntax, variables, data types, operators, conditions, loops, functions, arrays, objects, DOM, events, forms, JSON and browser storage.

Study notes with simple explanations and practical examples
22-page learning guide

1. Introduction to JavaScript

JavaScript is a programming language used to make web pages interactive. HTML creates the structure of a page, CSS controls its appearance, and JavaScript adds behaviour.

What can JavaScript do?

• Respond when a user clicks a button.

• Validate information entered in a form.

• Show or hide content without reloading the page.

• Update text, images and styles dynamically.

• Read and update data in the browser.

• Communicate with a server to retrieve or send information.

JavaScript in a web page

A browser can run JavaScript directly. A script can be written inside an HTML file or stored in a separate .js file.

```
<!DOCTYPE html>
<html>
<head>
  <title>My First JavaScript Page</title>
</head>
<body>
  <h1>Hello, web!</h1>
  <script>
    alert("Welcome to JavaScript");
  </script>
</body>
</html>
```

The alert() statement displays a small message box. In real projects, JavaScript is commonly kept in a separate file.

2. Adding JavaScript to HTML

Method 1: Internal JavaScript

Write JavaScript between opening and closing script tags in the HTML document.

```
<script>
  console.log("Hello from JavaScript");
</script>
```

Method 2: External JavaScript

Create a file named script.js and connect it to the HTML page. This keeps the structure and behaviour separate.

```
<!-- index.html -->
<!DOCTYPE html>
<html>
<body>
  <h2>JavaScript Example</h2>
  <script src="script.js" defer></script>
</body>
</html>

/* script.js */
console.log("External file connected");
```

The defer attribute tells the browser to run the external script after the HTML has been parsed. It is useful when the script needs to access page elements.

Where can output appear?

• console.log() prints information in the browser developer console.

• alert() displays a message box.

• The DOM can be used to display output directly on the page.

3. JavaScript Syntax and Statements

Syntax means the rules used to write valid code. A JavaScript program is made up of statements and expressions.

Important syntax rules

• JavaScript is case-sensitive: total and Total are different names.

• Use quotes around text values.

• Use parentheses with function calls.

• Use curly braces to group blocks of code.

• A semicolon can mark the end of a statement. It is recommended for clarity.

• Spaces and indentation make code easier to read.

```
let name = "Anu";
let age = 20;

if (age >= 18) {
  console.log(name + " is an adult.");
}
```

Comments

Comments explain code and are ignored when the program runs.

```
// This is a single-line comment

/*
  This is a
  multi-line comment
*/
```

Choose meaningful names and keep indentation consistent. Clear code is easier to test and maintain.

4. Variables: let, const and var

A variable is a named place used to store a value. JavaScript provides let, const and var for declaring variables.

let

Use let when the value may change later.

```
let score = 10;
score = 15;
console.log(score); // 15
```

const

Use const when the variable should not be reassigned.

```
const college = "ABC College";
console.log(college);
```

var

var is the older way to declare variables. It has function scope and can lead to confusing behaviour, so let and const are preferred in modern JavaScript.

```
var city = "Erode";
console.log(city);
```

Naming rules

• A name can contain letters, digits, underscore and dollar sign.

• A name cannot begin with a digit.

• Reserved keywords such as if and let cannot be used as variable names.

• Use descriptive names such as studentName or totalMarks.

5. Data Types

A data type describes the kind of value stored in a variable.

Common primitive data types

Type	Example	Meaning
String	"JavaScript"	Text
Number	25 or 4.5	Integer or decimal number
Boolean	true / false	Logical value
Undefined	let x;	Declared but not assigned
Null	let x = null;	Intentional empty value
BigInt	123n	Very large integer
Symbol	Symbol('id')	Unique identifier

Checking a type

```
let price = 99.50;
console.log(typeof price);   // "number"

let subject = "JavaScript";
console.log(typeof subject); // "string"
```

JavaScript also has objects and arrays, which are used to organise related data.

6. Operators

Operators perform actions on values and variables.

Arithmetic operators

```
let a = 10;
let b = 3;

console.log(a + b); // 13
console.log(a - b); // 7
console.log(a * b); // 30
console.log(a / b); // 3.333...
console.log(a % b); // 1 (remainder)
console.log(a ** b); // 1000
```

Comparison operators

Comparison operators return true or false. Examples: == (loose equality), === (strict equality), !=, !==, greater than, less than, greater than or equal to, and less than or equal to.

```
console.log(5 == "5");  // true
console.log(5 === "5"); // false
console.log(8 > 4);     // true
```

Logical operators

```
let age = 20;
let hasID = true;

console.log(age >= 18 && hasID); // true
console.log(age < 18 || hasID);  // true
console.log(!hasID);             // false
```

Prefer === and !== in most comparisons because they also check the data type.

7. Input and Output

Displaying output

```
console.log("Welcome");
alert("Hello!");
document.write("Page output");
```

console.log() is especially useful while learning and debugging. document.write() is generally avoided for updating an existing page.

Getting input with prompt()

```
let userName = prompt("Enter your name:");
console.log("Hello, " + userName);
```

prompt() returns text. Convert it when you need to perform calculations.

```
let first = Number(prompt("Enter first number:"));
let second = Number(prompt("Enter second number:"));

let sum = first + second;
alert("Sum = " + sum);
```

String concatenation

The + operator joins strings. Template literals, written with backticks, make it easier to combine text and variables.

```
let name = "Ravi";
let mark = 85;

console.log("Student: " + name);
console.log(`Student: ${name}, Mark: ${mark}`);
```

8. Conditional Statements

Conditional statements allow a program to make decisions.

if statement

```
let mark = 75;

if (mark >= 50) {
  console.log("Pass");
}
```

if...else

```
let number = 8;

if (number % 2 === 0) {
  console.log("Even number");
} else {
  console.log("Odd number");
}
```

else if ladder

```
let mark = 82;

if (mark >= 90) {
  console.log("Grade A");
} else if (mark >= 75) {
  console.log("Grade B");
} else if (mark >= 50) {
  console.log("Grade C");
} else {
  console.log("Fail");
}
```

Conditions are evaluated from top to bottom. The first true block is executed.

9. switch Statement and Ternary Operator

switch statement

Use switch when one expression is compared with several possible values.

```
let day = 3;

switch (day) {
  case 1:
    console.log("Monday");
    break;
  case 2:
    console.log("Tuesday");
    break;
  case 3:
    console.log("Wednesday");
    break;
  default:
    console.log("Invalid day");
}
```

break stops execution from continuing into the next case. default runs when no case matches.

Ternary operator

The ternary operator is a short form of a simple if...else statement.

```
let age = 19;
let result = age >= 18 ? "Eligible" : "Not eligible";
console.log(result);
```

Use the ternary operator for short decisions. Use if...else when the logic is longer or has multiple steps.

10. Loops

Loops repeat a block of code while a condition is satisfied.

for loop

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

The for loop has three parts: initialisation (let i = 1), condition (i <= 5), and update (i++).

while loop

```
let i = 1;

while (i <= 5) {
  console.log(i);
  i++;
}
```

do...while loop

```
let i = 1;

do {
  console.log(i);
  i++;
} while (i <= 5);
```

A do...while loop executes its body at least once because the condition is checked at the end.

11. Loop Control and Nested Loops

break

break immediately ends the nearest loop.

```
for (let i = 1; i <= 10; i++) {
  if (i === 5) {
    break;
  }
  console.log(i);
}
// Prints 1, 2, 3, 4
```

continue

continue skips the current iteration and moves to the next one.

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) {
    continue;
  }
  console.log(i);
}
// Prints 1, 2, 4, 5
```

Nested loop

A nested loop is a loop inside another loop. It is useful for patterns and two-dimensional data.

```
for (let row = 1; row <= 3; row++) {
  let line = "";
  for (let col = 1; col <= 3; col++) {
    line += "* ";
  }
  console.log(line);
}
```

12. Functions

A function is a reusable block of code designed to perform a task.

Function declaration

```
function greet() {
  console.log("Welcome!");
}

greet(); // function call
```

Parameters and arguments

```
function greetUser(name) {
  console.log("Hello, " + name);
}

greetUser("Meena");
```

name is a parameter in the function definition. Meena is an argument supplied when the function is called.

Return value

```
function add(a, b) {
  return a + b;
}

let answer = add(10, 20);
console.log(answer); // 30
```

return sends a value back to the code that called the function. It also ends that function's execution.

13. Arrow Functions and Scope

Arrow function

Arrow functions provide a shorter way to write function expressions.

```
const multiply = (a, b) => {
  return a * b;
};

console.log(multiply(4, 5)); // 20

const square = n => n * n;
console.log(square(6)); // 36
```

Variable scope

Scope determines where a variable can be accessed.

• Global scope: declared outside functions or blocks and available in its scope.

• Function scope: a var variable declared inside a function is available within that function.

• Block scope: let and const declared inside { } are available only inside that block.

```
if (true) {
  let message = "Inside block";
  console.log(message);
}
// console.log(message); // Error
```

Prefer let and const and declare variables close to where they are used.

14. Arrays

An array stores multiple values in one variable. Array positions are called indexes, and indexing starts at 0.

```
let fruits = ["Apple", "Mango", "Orange"];

console.log(fruits[0]); // Apple
console.log(fruits[2]); // Orange
console.log(fruits.length); // 3
```

Common array methods

• push(value): adds an item to the end.

• pop(): removes the last item.

• unshift(value): adds an item at the beginning.

• shift(): removes the first item.

• includes(value): checks whether an item exists.

• indexOf(value): returns the first matching index, or -1.

```
let fruits = ["Apple", "Mango"];
fruits.push("Orange");
fruits.pop();
fruits.unshift("Banana");
console.log(fruits);
```

Use a loop to process every item in an array.

15. Array Iteration and Useful Methods

for...of loop

```
let marks = [70, 80, 90];

for (let mark of marks) {
  console.log(mark);
}
```

forEach()

```
marks.forEach(function(mark) {
  console.log(mark);
});
```

map()

map() creates a new array by transforming each item.

```
let numbers = [1, 2, 3];
let doubled = numbers.map(n => n * 2);

console.log(doubled); // [2, 4, 6]
```

filter()

filter() creates a new array containing items that pass a condition.

```
let ages = [12, 18, 21, 15];
let adults = ages.filter(age => age >= 18);

console.log(adults); // [18, 21]
```

map() changes each item into a new result; filter() selects items that satisfy a test.

16. Objects

An object stores related information as key-value pairs. The keys are also called properties.

```
let student = {
  name: "Kavi",
  age: 20,
  course: "MCA"
};

console.log(student.name);
console.log(student["course"]);
```

Changing and adding properties

```
student.age = 21;
student.city = "Erode";

console.log(student);
```

Object methods

A function stored as an object property is called a method.

```
let person = {
  name: "Arun",
  greet: function() {
    console.log("Hello, " + this.name);
  }
};

person.greet();
```

The this keyword refers to the object that is calling the method in this example.

17. DOM: Document Object Model

The DOM represents an HTML page as a tree of objects. JavaScript can use the DOM to read and change page content, attributes and styles.

Selecting elements

```
document.getElementById("title");
document.getElementsByClassName("item");
document.getElementsByTagName("p");
document.querySelector(".message");
document.querySelectorAll("p");
```

Changing content

```
// HTML: <h2 id="title">Old title</h2>

const heading = document.getElementById("title");
heading.textContent = "New title";
```

Changing styles

```
heading.style.color = "blue";
heading.style.fontSize = "28px";
```

textContent is a safe and straightforward choice for inserting plain text. Use innerHTML only when you intentionally need to insert HTML and have handled untrusted input safely.

18. Events

An event is an action or occurrence detected by the browser, such as a click, key press, page load or form submission.

Handling a click

```
<!-- HTML -->
<button id="btn">Click me</button>
<p id="output"></p>

// JavaScript
const button = document.getElementById("btn");
const output = document.getElementById("output");

button.addEventListener("click", function() {
  output.textContent = "Button clicked!";
});
```

Common events

• click: user clicks an element.

• input: value in an input field changes.

• change: a form control's value is committed.

• submit: a form is submitted.

• mouseover: pointer moves over an element.

• keydown: a keyboard key is pressed.

addEventListener() is commonly used because it keeps JavaScript separate from HTML.

19. Forms and Basic Validation

Forms collect information such as a name, email address or password. HTML attributes can provide simple built-in validation.

```
<form id="registerForm">
  <label>Name:</label>
  <input type="text" id="name" required>

  <label>Email:</label>
  <input type="email" id="email" required>

  <button type="submit">Register</button>
</form>
<p id="message"></p>
```

Handling form submission

```
const form = document.getElementById("registerForm");
const message = document.getElementById("message");

form.addEventListener("submit", function(event) {
  event.preventDefault();

  const name = document.getElementById("name").value;
  message.textContent = `Welcome, ${name}!`;
});
```

event.preventDefault() stops the browser's default form submission so the script can handle the event. required and type=email provide basic browser validation.

Important: client-side validation improves user experience, but a server must validate important data again.

20. JSON and Browser Storage

JSON

JSON stands for JavaScript Object Notation. It is a text format commonly used to exchange data between a browser and a server.

```
const student = {
  name: "Nila",
  course: "MCA"
};

const jsonText = JSON.stringify(student);
console.log(jsonText);

const objectAgain = JSON.parse(jsonText);
console.log(objectAgain.name);
```

JSON.stringify() converts a JavaScript value into JSON text. JSON.parse() converts valid JSON text into a JavaScript value.

localStorage and sessionStorage

• localStorage keeps data in the browser until it is cleared.

• sessionStorage keeps data for the current browser tab session.

• Both store values as strings, so JSON.stringify() and JSON.parse() are useful for objects.

```
// Save
localStorage.setItem("studentName", "Nila");

// Read
let savedName = localStorage.getItem("studentName");

// Remove one item
localStorage.removeItem("studentName");
```

Do not store passwords or sensitive information in browser storage.

21. Debugging, Best Practices and Practice Tasks

Debugging basics

• Read the error message in the browser console.

• Check spelling, brackets, quotes and semicolons.

• Use console.log() to inspect variable values.

• Check whether the JavaScript file is connected correctly.

• Use the browser developer tools to inspect HTML, CSS and JavaScript.

Good coding practices

• Use const by default and let when reassignment is needed.

• Choose meaningful variable and function names.

• Indent code consistently.

• Break large tasks into small functions.

• Avoid repeating the same code unnecessarily.

• Test normal, empty and invalid inputs.

Beginner practice tasks

• Display your name and course using console.log().

• Read two numbers and display their sum.

• Check whether a number is even or odd.

• Print numbers from 1 to 10 using a for loop.

• Find the largest value in an array.

• Create a student object and display its properties.

• Change a heading when a button is clicked.

• Create a simple form that displays a welcome message.

Quick recap

JavaScript adds behaviour to web pages. Start with variables, data types, operators, conditions and loops. Then practise functions, arrays, objects, DOM selection, events, forms, JSON and browser storage. Small working examples are the best way to build confidence.